Application note 008

Beginner's guide to calculate programmed sums, demonstrated with Ramanujan's famous series for the inverse of **π**, and the C47 Σ_n function:

$$\frac{1}{\pi} = \frac{\sqrt{8}}{9801} \sum_{n=0}^{+\infty} \frac{(4n)!}{(n!)^4} \frac{1103 + 26390n}{396^{4n}}$$

*S. Ramanujan **

$$\boxed{\frac{(4n)!}{(n!)^4} \frac{1103 + 26390n}{396^{4n}}}$$

Repeating/iterative section

Start by setting up the repeating C47 RPN program for the sum (the keystrokes are detailed in the annexure below):

Create a new program "SR", the iterative section of the Ramanujan series:

Create new program: `g[GTO].. f[PRGM]`

Label it: `g[LBL]` a `S R [ENTER]`

Now create the RPN function: (Tip: When using the programmed sum function, for each internal iteration the stack will be pre-filled with n, allowing extensive RPN stack manipulation with **x⇄, y⇄, z⇄, t⇄, x⇄y, RCL T,** etc.).

Enter it:

```
4 × [x!] [x⇄y]
[x!] 4 [y^x] ÷ [x⇄y]
4 × 396 [x⇄y] [y^x] ÷ [x⇄y]
26390 × 1103 + ×
[RTN]
```

Program listing:

```
C47 Program file export: Export format version 2, C47 program version 1.
0000: { Prgm #42: 56 bytes / 24 steps }

0001:  LBL 'SR'
0002:    4
0003:    *
0004:    X!
0005:    X<>Y
0006:    X!
0007:    4
0008:    Y^X
0009:    /
0010:    X<>Y
0011:    4
0012:    *
0013:    396
0014:    X<>Y
0015:    Y^X
0016:    /
0017:    X<>Y
0018:    26 390
0019:    *
0020:    1 103
0021:    +
0022:    *
0023:  RTN
0024:  END
```

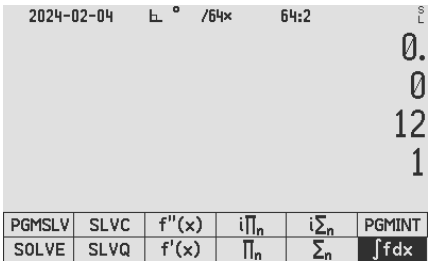
Since clearly the C47 cannot and should not do infinite iterations of this sum ;-) we need to determine how many iterations are needed for the accuracy required, but let's first do the example below for N = 12, then do more targeted analysis later:

Do the programmed sum as follows: From 0 to 12 in steps of 1, do Σ_n as follows:

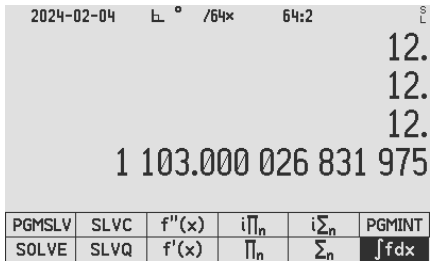
Programmed sum: **0 [ENTER] 12 [ENTER] 1 [Σ_n] 'SR'**

Tip: find [Σ_n] on the [ADV] menu:

The result on the stack, is the programmed sum for the first 13 terms:

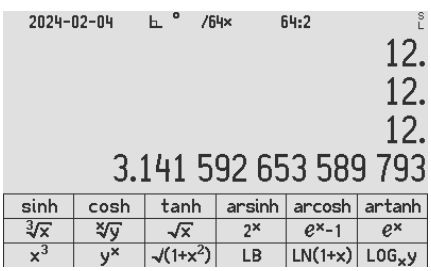


Ready to execute Σ_n



After execution of Σ_n

After completing Ramanujan's formula: **8 [√x] × 9801 ÷ [1/x],** the result seems very close to **π**.



Ramanujan's result

To determine the distance from the built-in 34-digit **π**, we subtract **π** from our Ramanujan result with the result of **0.** indicating that the 13 terms used resulted in the same accuracy as our stored **π** constant.

An interesting question is what is the minimum number of terms needed for the Ramanujan series to be accurate to compare with the internal C47 rendition of **π**?

An intuitive inspection of the formula reveals that we are dealing with the largest factors being (4n)! and 396^{4n}. Noting that precision is maintained to 34 digits on C47, it can be deduced that for the factor (4n)!, (4x12)! > 1×10^{34} could be a limit, and for 396^{4n}, $396^{4x4} > 1\times10^{34}$ could be a limit. This suggests that values of n, where n > 4 could result in programmed sum increments that cannot be registered in our 34-digit C47 floating point precision.

To check this hypothesis, we can do this procedure for multiple values of n to verify how many terms are needed for the accuracy level required, and to do this comfortably, we can make a second program, to sequence the previous programmed sum steps as follows:

Make a new program called SRSEQ which will calculate the programmed sum for 0 to n, complete the Ramanujan formula to obtain for Ramanujan's **π** (which I call *R(**π**)*) below, and subtract the C47 **π** constant from it, resulting in: *R(**π**)* - C47(**π**) in X. This new program is will be used in the form "`n SRSEQ`" which will result in a single accuracy figure remaining in X:

Create a new program "SRSEQ", to sequence the previous "Sum of SR" Ramanujan series:

Create New program:	`g[GTO].. f[PRGM]`
Label it:	`g[LBL] a S R S E Q [ENTER]`
Enter it:	`0 [STO] Z [R↓] 1 [Σn] 'SR' 8 √x × 9801 ÷ [1/x] π - f[PRGM]`
Run it:	`3 [XEQ] [PROG] [SRSEQ]`

```
C47 Program file export: Export format version 2, C47 program version 1.
0000: { Prgm #44: 44 bytes / 15 steps }
0001:  LBL 'SRSEQ'
0002:    0
0003:    STO Z
0004:    Rv
0005:    1
0006:    SUMn 'SR'
0007:    8
0008:    SQRT
0009:    *
0010:    9 801
0011:    /
0012:    1/X
0013:    PI
0014:    -
0015:  .END.
```

This shows that the accuracy that the Ramanujan **π** converges extremely fast to **π**:

N	Terms	Proximity to **π** = [1 / (SR(N) × √8 / 9801)] - **π**
0 SRSEQ	1	7.6423512421851352805745713E-8
1 SRSEQ	2	6.39536262443026511E-16
2 SRSEQ	3	5.682423256E-24
3 SRSEQ	4	5.1E-32
4 SRSEQ	5	0.

It is shown in the table that the initial guess of n > 4, sufficiently converges to 34 digits accuracy.

Detailed keystrokes to make a new program & label it:

g[GTO].. f[PRGM]	α GTO XEQ F · SHOW INFO · SHOW INFO · PRGM TEST R/S ?
g[LBL] *a* SR [ENTER]	e^x LBL LN E · α GTO XEQ F · STAT PLOT ÷ S · MATX X.FN 9 R · COMPLEX CPX ENTER

Detailed keystrokes to enter the "SR" program:

4 × [x!] [x⇄y]	4 PROB INTS × W · PROB INTS × W · x! · LASTx STK x⇄y M
[x!] 4 [y^x] ÷ [x⇄y]	PROB INTS × W · x! · 4 · y^x # 1/x B · STAT PLOT ÷ S · LASTx STK x⇄y M
4 × 396 [x⇄y] [y^x] ÷ [x⇄y]	4 PROB INTS × W · 396 · LASTx STK x⇄y M · y^x # 1/x B · STAT PLOT ÷ S · LASTx STK x⇄y M
26390 × 1103 + ×	26390 PROB INTS × W · 1103 · CAT CNST + ␣ · PROB INTS × W
[RTN]	P.FN LOOP 3 Z · RTN

Detailed keystrokes to enter the "SRSEQ" program (after SRSEQ created in PRGM mode):

0 [STO] Z [R↓] 1 [Σ_n] 'SR'	0 |x| STO G · Z π $\sqrt[x]{y}$ R↓ I · 1 · ADV FIN 8 Q · PROG 'SR'
8 √x × 9801 ÷ [1/x]	8 x^2.ms √x C · PROB INTS × W · 9801 · STAT PLOT ÷ S · y^x # 1/x B · π -
f[PRGM]	PRGM TEST R/S ?

Executing a program SRSEQ:

3 [XEQ] [PROG] [SRSEQ]	3 α GTO XEQ F · PROG · STAT113 STAT115 STATgr X Y Z / SR1 SRSEQ STAT000 STAT001 STAT002 STAT111 / Satell SlvCnst SlvExtr Speed SPIRAL SR

* Source for the example: Ramanujan, as discussed here: https://arxiv.org/pdf/2104.12412.pdf

Author Jaco Mostert

Copyright (C) 2024 The C47 Development Team. Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license can be downloaded from gnu.org.